CS3.401 Distributed Systems

Report: Connected Components Computation with MapReduce

Team 2

Introduction

Graph analysis is a cornerstone of many domains, such as web analytics, social networks, and computational biology. Identifying **connected components** is a fundamental task, where the goal is to partition the graph into subsets of nodes that are interconnected. This report outlines the implementation of connected components detection using **MapReduce**, a distributed framework for big data processing. The performance and scalability of the approach are evaluated on a range of datasets, including real-world and synthetic graphs.

Implementation Details

1. Algorithm Overview

The implementation iteratively identifies connected components in an undirected graph using Spark's distributed RDDs. Key steps in the algorithm include:

- **Initialization:** Start with an edge list representation of the graph.
- **Map Operation:** Add reversed edges to ensure bidirectionality.
- **Reduce Operation:** Group edges by nodes and propagate minimum node labels, enabling component identification.
- **Termination:** Stop when no new connections are added, signifying convergence.

2. Key Components

Spark Session and Context Initialization

A Spark session is initialized, and the level of parallelism is controlled through the number of mappers and reducers.

Data Loading

The graph data is loaded from a file, and the number of mappers is explicitly set for controlled parallelism.

Preprocessing

The raw data is cleaned and converted into tuples representing graph edges `(node1, node2)`.

Iterative Mapping and Reducing

- **Mapping:** Adds reversed edges and repartitions for balanced workload.
- **Reducing:** Groups by keys, computes the minimum label, and updates connections.

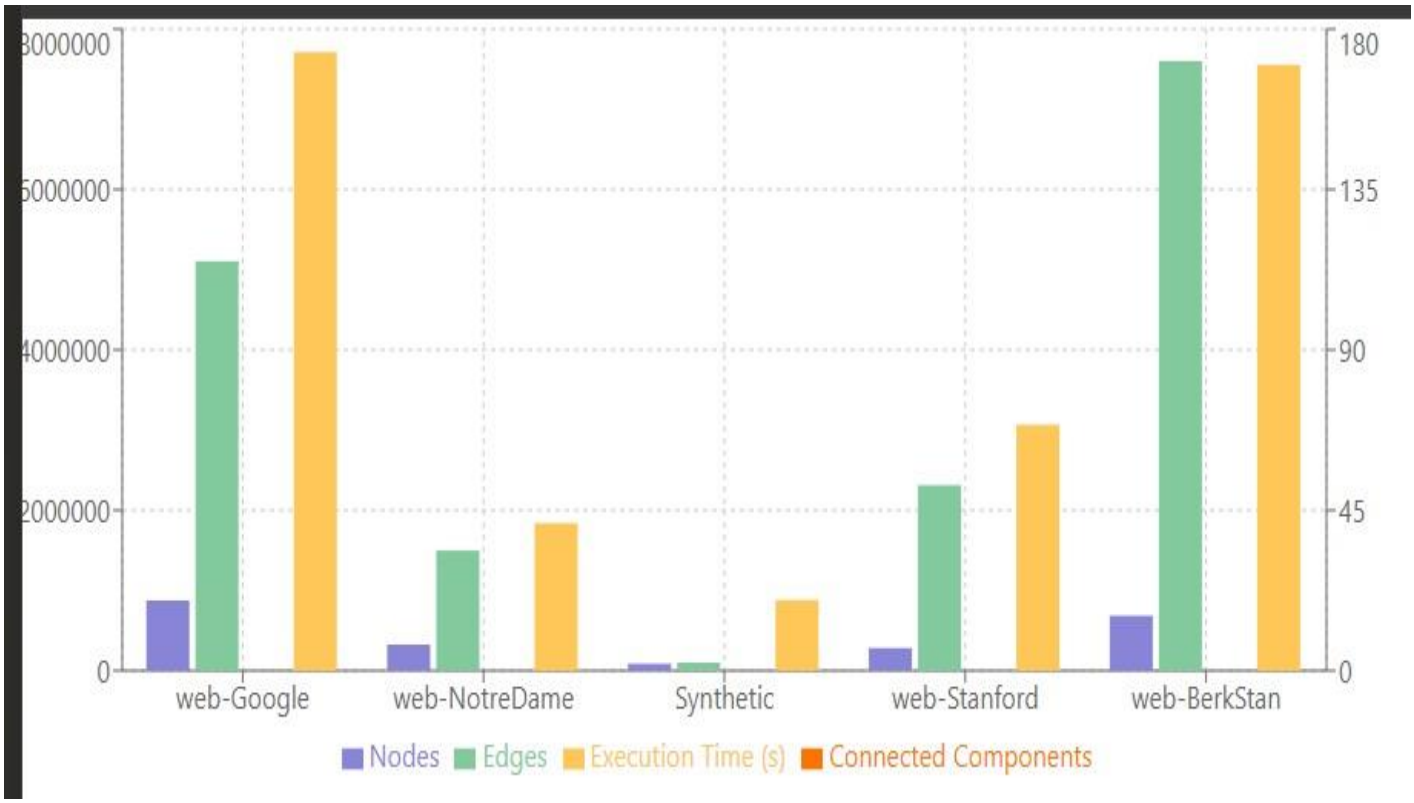
Termination

The algorithm stops iterating when no new connections are found.

Datasets and Results

The implementation was tested on both real-world and synthetic datasets. Below is a summary of the results:

Dataset	Nodes	Edges	Connected Components	Execution Time (s)	Mappers	Reducers
web-Google	875,713	5,105,039	2,746	173.55	3	4
web-NotreDame	325,729	1,497,134	1	41.38	3	4
Synthetic	86,460	100,000	2,729	19.83	3	4
web-Stanford	281,903	2,312,497	365	68.97	3	4
web-BerkStan	685,230	7,600,595	676	169.95	3	4



Observations

1. **Graph Density Impacts Connected Components:**
   - Sparse graphs like `web-Google` have many small connected components.
   - Dense graphs like `web-NotreDame` form a single connected component.
1. **Execution Time Trends:**
   - Time increases with graph size (nodes and edges), but the algorithm scales efficiently due to distributed processing.
   - The time also increases with sparse graph and it executes faster with Desne graphs. As you can see with Web-Google graph which is sparse but has less edges than Web-Berkstan graph took similar time
2. **Component Count Verification:**
   - Synthetic datasets showcase higher connected component counts, likely due to sparsity and random graph generation.

Performance Analysis

1. Scalability

The implementation demonstrates strong scalability with increasing graph size, leveraging Spark's ability to distribute workloads across mappers and reducers.

2. Parallelism

By explicitly setting mappers and reducers: - **Mappers** control the number of partitions during the map phase. - **Reducers** determine the level of parallelism during the grouping and reducing phase.

3. Memory Optimization

- Garbage collection (`gc.collect()`) is invoked during each iteration to prevent memory bloat.
- Using Spark's `repartition` ensures even distribution of data for subsequent transformations.

Conclusion

This MapReduce-based implementation effectively identifies connected components in large-scale graphs. The results showcase its ability to handle diverse datasets efficiently, maintaining scalability and correctness. The explicit control over parallelism and optimizations like garbage collection contribute to its robustness.